```
import os
import time
import json
import queue
import audioop
import statistics
import sounddevice as sd
from vosk import Model, KaldiRecognizer
from rplidar import RPLidar

# ============================================================
# 1. SETTINGS
# ============================================================

MODEL_PATH = "/home/pi/vision_ws/vosk-model-small-en-us-0.15"

# USB mic
MIC_INDEX = 1
MIC_SAMPLE_RATE = 44100
VOSK_SAMPLE_RATE = 16000

# LiDAR port
LIDAR_PORT = "/dev/ttyUSB0"

# Location confidence threshold
LOCATION_SCORE_THRESHOLD = 260

# Best-vs-second-best score gap
LOCATION_SCORE_GAP_THRESHOLD = 25

# Obstacle thresholds in mm
FRONT_ALERT_DISTANCE = 700
SIDE_ALERT_DISTANCE = 500
ALERT_COOLDOWN = 3.0

# ============================================================
# 2. LOCATION SIGNATURES
#    Added "back" to improve discrimination
#    Update these later with real measured values if needed
# ============================================================

KNOWN_LOCATIONS = {
    "eldc lab": {"front": 483, "left": 111, "right": 467},
    "automation lab": {"front": 700, "left": 260, "right": 240},
    "robotics lab": {"front": 398, "left": 1291, "right": 983},
    "male washroom": {"front": 300, "left": 200, "right": 150},
}

# ============================================================
# 3. LOCATION ALIASES
# ============================================================

location_aliases = {
    "automation lab": [
        "automation lab",
        "automation laboratory",
        "auto lab","photo lab"," auto",""
    ],
    "robotics lab": [
        "robotics lab",
        "robotics laboratory",
        "bot lab",
```

```
        "boutiquescation",
        "robot",
        "robo","human"
    ],
    "eldc lab": [
        "eldc lab",
        "eldc laboratory",
        "eldc",
        "electronics",
        "electronicstion",
        "ldc lablocation"
    ],
    "male washroom": [
        "male washroom",
        "mens washroom",
        "men washroom",
        "mens room",
        "mens toilet",
        "male restroom",
        "gents washroom",
        "gents toilet"
    ]
}

# ============================================================
# 4. ROUTES DATABASE
# ============================================================

routes_db = {
    ("automation lab", "robotics lab"): [
        {"action": "turn", "direction": "right"},
        {"action": "move", "steps": 48},
        {"action": "turn", "direction": "right"},
        {"action": "arrive", "message": "You have reached Robotics Lab."}
    ],

    ("robotics lab", "eldc lab"): [
        {"action": "turn", "direction": "right"},
        {"action": "move", "steps": 76},
        {"action": "turn", "direction": "right"},
        {"action": "arrive", "message": "You have reached ELDC Lab."}
    ],

    ("eldc lab", "robotics lab"): [
        {"action": "turn", "direction": "left"},
        {"action": "move", "steps": 76},
        {"action": "turn", "direction": "left"},
        {"action": "arrive", "message": "You have reached Robotics Lab."}
    ],

    ("automation lab", "eldc lab"): [
        {"action": "turn", "direction": "right"},
        {"action": "move", "steps": 124},
        {"action": "turn", "direction": "right"},
        {"action": "arrive", "message": "You have reached ELDC Lab."}
    ],

    ("eldc lab", "automation lab"): [
        {"action": "turn", "direction": "left"},
        {"action": "move", "steps": 124},
        {"action": "turn", "direction": "left"},
        {"action": "arrive", "message": "You have reached Automation Lab."}
```

```
    ],

    ("eldc lab", "male washroom"): [
        {"action": "move", "steps": 20},
        {"action": "stairs", "message": "Go down a few staircases carefully."},
        {"action": "move", "steps": 13},
        {"action": "deviate", "angle": 120, "direction": "left"},
        {"action": "move", "steps": 15},
        {"action": "turn", "direction": "left"},
        {"action": "move", "steps": 5},
        {"action": "arrive", "message": "You have reached the men's washroom."}
    ]
}

# ============================================================
# 5. TRAVEL TIME DATABASE
# ============================================================

travel_time_db = {
    ("automation lab", "eldc lab"): 60,
    ("eldc lab", "automation lab"): 60,
    ("automation lab", "robotics lab"): 15,
    ("robotics lab", "automation lab"): 15,
    ("robotics lab", "eldc lab"): 45,
    ("eldc lab", "robotics lab"): 45
}

# ============================================================
# 6. GLOBALS
# ============================================================

audio_queue = queue.Queue()
resample_state = None

obstacle_detection_enabled = False
last_alert_time = 0
last_alert_message = ""

lidar_global = None

# ============================================================
# 7. SPEAK
# ============================================================

def speak(text):
    print(f"Assistant: {text}")
    safe_text = text.replace('"', '\\"')
    os.system(f'espeak -s 140 "{safe_text}" 2>/dev/null')

# ============================================================
# 8. LIDAR INTERFACE
# ============================================================

class LidarInterface:
    def __init__(self):
        self.lidar = RPLidar(LIDAR_PORT)
        print("[SYSTEM] LiDAR connected")

    def _sector_values(self, scan, center, width=40):
        vals = []
        start_angle = (center - width) % 360
        end_angle = (center + width) % 360
```

```
        for _, angle, distance in scan:
            if distance <= 0:
                continue

            if start_angle <= end_angle:
                if start_angle <= angle <= end_angle:
                    vals.append(distance)
            else:
                if angle >= start_angle or angle <= end_angle:
                    vals.append(distance)

        return vals

    def _robust_value(self, vals):
        if not vals:
            return None
        vals = sorted(vals)
        if len(vals) >= 5:
            vals = vals[1:-1]
        return int(statistics.median(vals))

    def get_reading(self):
        try:
            for scan in self.lidar.iter_scans():
                front_vals = self._sector_values(scan, center=0, width=40)
                left_vals = self._sector_values(scan, center=90, width=40)
                right_vals = self._sector_values(scan, center=270, width=40)
                back_vals = self._sector_values(scan, center=180, width=40)

                reading = {
                    "front": self._robust_value(front_vals),
                    "left": self._robust_value(left_vals),
                    "right": self._robust_value(right_vals),
                    "back": self._robust_value(back_vals),
                }

                print(f"[LIDAR LIVE] {reading}")
                return reading

        except Exception as e:
            print(f"[LIDAR ERROR] {e}")
            return {"front": None, "left": None, "right": None, "back": None}

    def stop(self):
        try:
            self.lidar.stop()
        except Exception:
            pass
        try:
            self.lidar.disconnect()
        except Exception:
            pass

# ============================================================
# 9. LOCATION NORMALIZATION
# ============================================================

def normalize_place_name(name):
    if not name:
        return None
```

```
    name = name.strip().lower()

    if name in KNOWN_LOCATIONS:
        return name

    for canonical_name, aliases in location_aliases.items():
        for alias in aliases:
            if alias == name:
                return canonical_name

    for canonical_name, aliases in location_aliases.items():
        for alias in aliases:
            if alias in name:
                return canonical_name

    return None

# ============================================================
# 10. INTENT DETECTION
# ============================================================

def detect_intent(text):
    if text is None:
        return "NONE"

    text = text.lower().strip()
    words = text.split()

    start_phrases = [
        "start", "begin", "go", "start navigation", "okay", "ok", "yes",
        "confirm", "confirmed", "continue", "proceed", "star", "art"
    ]
    exit_phrases = [
        "exit", "quit", "stop", "stop program", "shutdown",
        "bye", "bi", "by", "close"
    ]

    if text in start_phrases:
        return "START"
    if text in exit_phrases:
        return "EXIT"

    for word in words:
        if word in ["start", "begin", "go", "ok", "okay", "yes", "confirm", "continue",
"proceed", "star", "art"]:
            return "START"
        if word in ["exit", "quit", "stop", "shutdown", "bye", "bi", "by", "close"]:
            return "EXIT"

    if text in ["o k", "okayy", "stark", "stat", "estate"]:
        return "START"
    if text in ["eggs it", "ex it", "quite", "stoop"]:
        return "EXIT"

    return "UNKNOWN"

# ============================================================
# 11. LOCALIZATION
# ============================================================

def get_lidar_signature(lidar, samples=10):
    readings = []
```

```
    print("[SYSTEM] Collecting current location signature...")
    for i in range(samples):
        reading = lidar.get_reading()
        print(f"Reading {i+1}: {reading}")
        readings.append(reading)
        time.sleep(0.2)

    avg = {"front": None, "left": None, "right": None, "back": None}

    for direction in avg:
        vals = [r[direction] for r in readings if r[direction] is not None]
        if vals:
            avg[direction] = round(sum(vals) / len(vals), 2)

    return avg


def compare_one_mapping(current, saved):
    score = 0

    for key in ["front", "left", "right", "back"]:
        c = current.get(key)
        s = saved.get(key)

        if c is None or s is None:
            score += 500
            continue

        diff = abs(c - s)
        norm = diff / max(s, 1)
        score += norm * 100

    return round(score, 2)


def compare_signatures(current, saved):
    mappings = [
        saved,
        {
            "front": saved["left"],
            "left": saved["back"],
            "right": saved["front"],
            "back": saved["right"]
        },
        {
            "front": saved["right"],
            "left": saved["front"],
            "right": saved["back"],
            "back": saved["left"]
        },
        {
            "front": saved["back"],
            "left": saved["right"],
            "right": saved["left"],
            "back": saved["front"]
        },
    ]

    scores = [compare_one_mapping(current, mapping) for mapping in mappings]
    return min(scores)
```

```
def estimate_location(lidar, known_locations):
    current_signature = get_lidar_signature(lidar)
    print(f"[SYSTEM] Current signature: {current_signature}")

    results = []

    for loc_name, saved_signature in known_locations.items():
        print(f"[DEBUG] Saved signature for {loc_name}: {saved_signature}")
        score = compare_signatures(current_signature, saved_signature)
        print(f"[DEBUG] Score for {loc_name}: {score}")
        results.append((loc_name, score))

    results.sort(key=lambda x: x[1])

    best_location, best_score = results[0]
    second_location, second_score = results[1] if len(results) > 1 else (None, float("inf"))

    print(f"[DEBUG] Best: {best_location} -> {best_score}")
    print(f"[DEBUG] Second: {second_location} -> {second_score}")

    return best_location, best_score, second_location, second_score, current_signature


def get_current_location(lidar):
    best_location, best_score, second_location, second_score, signature =
estimate_location(lidar, KNOWN_LOCATIONS)
    print(f"[SYSTEM] Signature: {signature}")

    if best_location is None:
        speak("I could not estimate your location automatically")
        return None

    score_gap = second_score - best_score
    print(f"[DEBUG] Score gap: {score_gap}")

    if best_score < LOCATION_SCORE_THRESHOLD and score_gap > LOCATION_SCORE_GAP_THRESHOLD:
        print(f"[SYSTEM] Estimated location: {best_location}, score={best_score}")
        speak(f"Your current location is {best_location}")
        return best_location

    print(f"[WARNING] Location is ambiguous. Best match: {best_location}, score={best_score},
second={second_location}, second_score={second_score}")
    speak("I could not estimate your location automatically")
    return None

# ============================================================
# 12. OBSTACLE DETECTION
# ============================================================

def detect_obstacle_message(reading):
    front = reading.get("front")
    left = reading.get("left")
    right = reading.get("right")

    if front is not None and front < FRONT_ALERT_DISTANCE:
        return "Obstacle ahead"
    if left is not None and left < SIDE_ALERT_DISTANCE:
        return "Obstacle on left"
    if right is not None and right < SIDE_ALERT_DISTANCE:
        return "Obstacle on right"
```

```
    return None


def obstacle_monitor_step(lidar):
    global last_alert_time, last_alert_message

    if not obstacle_detection_enabled:
        return

    try:
        reading = lidar.get_reading()
    except Exception as e:
        print(f"[OBSTACLE ERROR] {e}")
        return

    if not reading:
        print("[OBSTACLE] No reading received")
        return

    print(f"[OBSTACLE] {reading}")

    front = reading.get("front")
    left = reading.get("left")
    right = reading.get("right")

    message = None

    if front is not None and front < FRONT_ALERT_DISTANCE:
        message = "Obstacle ahead"
    elif left is not None and left < SIDE_ALERT_DISTANCE:
        message = "Obstacle on left"
    elif right is not None and right < SIDE_ALERT_DISTANCE:
        message = "Obstacle on right"

    now = time.time()

    if message is not None:
        if message != last_alert_message or (now - last_alert_time) > ALERT_COOLDOWN:
            speak(message)
            last_alert_message = message
            last_alert_time = now

# ============================================================
# 13. AUDIO + LISTEN
# ============================================================

def audio_callback(indata, frames, time_info, status):
    global resample_state

    if status:
        status_text = str(status).lower()
        if "overflow" not in status_text:
            print(f"[AUDIO STATUS] {status}")

    raw_bytes = bytes(indata)

    converted, resample_state = audioop.ratecv(
        raw_bytes,
        2,
        1,
        MIC_SAMPLE_RATE,
        VOSK_SAMPLE_RATE,
```

```
        resample_state
    )

    audio_queue.put(converted)


def listen(model, prompt="Speak now", listen_seconds=6):
    global resample_state

    print(f"\n🎤 {prompt.upper()}...\n")

    while not audio_queue.empty():
        try:
            audio_queue.get_nowait()
        except queue.Empty:
            break

    resample_state = None
    rec = KaldiRecognizer(model, VOSK_SAMPLE_RATE)

    with sd.RawInputStream(
        samplerate=MIC_SAMPLE_RATE,
        blocksize=4096,
        device=MIC_INDEX,
        dtype="int16",
        channels=1,
        callback=audio_callback,
        latency="high"
    ):
        start_time = time.time()

        while True:
            if time.time() - start_time > listen_seconds:
                print("[WARNING] Listening timeout")
                return None

            try:
                data = audio_queue.get(timeout=0.5)
            except queue.Empty:
                continue

            if rec.AcceptWaveform(data):
                result = json.loads(rec.Result())
                text = result.get("text", "").strip().lower()

                if text:
                    print(f"[USER SAID] {text}")
                    return text

# ============================================================
# 14. ASK HELPERS
# ============================================================

def ask_until_valid_location(model):
    valid_names = list(KNOWN_LOCATIONS.keys())

    for _ in range(3):
        speak("Please say your current location")
        command = listen(model, prompt="Say current location", listen_seconds=6)

        print(f"[DEBUG] Raw spoken location: {command}")
```

```
        loc = normalize_place_name(command)

        print(f"[DEBUG] Normalized location: {loc}")
        print(f"[DEBUG] Valid names: {valid_names}")

        if loc in valid_names:
            speak(f"Current location set to {loc}")
            return loc

        speak("Location not recognized. Please say eldc lab, automation lab, robotics lab, or
male washroom")

    speak("Unable to get current location manually")
    return None


def ask_until_valid_destination(model):
    valid_names = list(KNOWN_LOCATIONS.keys())

    for _ in range(3):
        speak("Please say your destination")
        command = listen(model, prompt="Say destination", listen_seconds=6)
        dest = normalize_place_name(command)

        if dest in valid_names:
            speak(f"Destination selected as {dest}")
            return dest

        speak("Destination not recognized")

    speak("Unable to get destination")
    return None

# ============================================================
# 15. ROUTE SUMMARY + TIME MONITORING
# ============================================================

def speak_route_summary(start, destination, route):
    speak(f"Starting navigation from {start} to {destination}")

    for step in route:
        action = step.get("action")

        if action == "turn":
            msg = f"Turn {step.get('direction')}"
            print(f"[NAV] {msg}")
            speak(msg)

        elif action == "move":
            msg = f"Move forward {step.get('steps')} steps"
            print(f"[NAV] {msg}")
            speak(msg)

        elif action == "stairs":
            msg = step.get("message")
            print(f"[NAV] {msg}")
            speak(msg)

        elif action == "deviate":
            msg = f"Turn {step.get('angle')} degrees {step.get('direction')}"
            print(f"[NAV] {msg}")
            speak(msg)
```

```
def monitor_obstacles_for_duration(lidar, duration_seconds):
    start_time = time.time()
    speak(f"Estimated travel time is {duration_seconds} seconds")

    while time.time() - start_time < duration_seconds:
        obstacle_monitor_step(lidar)
        time.sleep(0.5)

# ============================================================
# 16. EXECUTE ROUTE
# ============================================================

def execute_route(start, destination):
    global obstacle_detection_enabled

    start = normalize_place_name(start)
    destination = normalize_place_name(destination)
    key = (start, destination)

    if key not in routes_db:
        print(f"[WARNING] Route not found for {start} to {destination}")
        speak("No route found for this destination")
        return False

    route = routes_db[key]

    print(f"[SYSTEM] Route map from {start} to {destination}:")
    for idx, step in enumerate(route, start=1):
        print(f"  Step {idx}: {step}")

    speak_route_summary(start, destination, route)

    obstacle_detection_enabled = True
    speak("Obstacle detection started")

    duration_seconds = travel_time_db.get(key, 10)
    monitor_obstacles_for_duration(lidar_global, duration_seconds)

    last_step = route[-1]
    if last_step.get("action") == "arrive":
        msg = last_step.get("message")
        print(f"[NAV] {msg}")
        speak(msg)

    obstacle_detection_enabled = False
    return True

# ============================================================
# 17. NAVIGATION FLOW
# ============================================================

def run_navigation_flow(model, lidar):
    current_location = get_current_location(lidar)

    if current_location is None:
        current_location = ask_until_valid_location(model)

    if current_location is None:
        speak("Navigation cancelled because current location is unknown")
        return
```

```
    destination = ask_until_valid_destination(model)

    if destination is None:
        speak("Navigation cancelled because destination is unknown")
        return

    if normalize_place_name(destination) == normalize_place_name(current_location):
        speak("You are already at the destination")
        return

    execute_route(current_location, destination)

# ============================================================
# 18. MAIN
# ============================================================

def main():
    global lidar_global

    print("[SYSTEM] Starting main system...")

    if not os.path.exists(MODEL_PATH):
        print(f"[ERROR] VOSK model not found at: {MODEL_PATH}")
        return

    model = Model(MODEL_PATH)
    lidar = LidarInterface()
    lidar_global = lidar

    speak("System started")
    speak("Say start or okay to begin navigation, or say exit to quit")

    running = True

    while running:
        command = listen(model, prompt="Say start or exit", listen_seconds=6)
        intent = detect_intent(command)

        if intent == "START":
            speak("Starting navigation")
            run_navigation_flow(model, lidar)
            speak("Say start or okay to begin navigation, or say exit to quit")

        elif intent == "EXIT":
            speak("Stopping the program")
            running = False

        elif intent == "NONE":
            speak("I did not hear anything clearly")

        else:
            speak("Please say start or okay to begin navigation, or say exit to quit")

    print("[SYSTEM] Program stopped")
    lidar.stop()

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\n[SYSTEM] Program interrupted by user")
```

```
        if lidar_global is not None:
            lidar_global.stop()
```